# MFAD-NET

*Multi-modal Fusion with Adaptive Drift Detection Network*

## Real-Time Social Media Monitoring System for Deepfake Detection

Major Project Report — First Review

Department of Computer Science & Engineering

2024 – 2025

# 1. Introduction

The rapid evolution of artificial intelligence has given rise to a new class of synthetic media commonly known as deepfakes. These are AI-generated or AI-manipulated images, videos, and audio clips that are crafted to be indistinguishable from authentic content. Powered by Generative Adversarial Networks (GANs), diffusion models, and autoencoder architectures, modern deepfake generation tools have reached a level of realism that makes visual detection by the human eye virtually impossible.

Social media platforms — including Instagram, Twitter/X, Facebook, YouTube, and WhatsApp — have become the primary distribution channels for deepfake content. The viral nature of these platforms accelerates the spread of manipulated media at a scale and speed that far outpaces human moderation capabilities. A single deepfake video can reach millions of viewers within hours of being posted, making real-time automated detection an urgent and critical need.

The consequences of undetected deepfakes are severe and wide-ranging. Political deepfakes have been used to fabricate statements by world leaders. Financial deepfakes have enabled CEO fraud and social engineering attacks. Non-consensual deepfake pornography has caused severe psychological harm to victims. Identity theft using deepfake faces has undermined biometric security systems. In each case, the damage compounds with time — making speed of detection as important as accuracy.

This project presents MFAD-Net (Multi-modal Fusion with Adaptive Drift Detection Network), a novel system designed specifically for real-time deepfake detection in social media environments. Unlike existing solutions that analyze only images or videos in isolation, MFAD-Net simultaneously processes visual frames, audio signals, and post metadata through a unified cross-modal architecture. Its defining innovation — the Temporal-Semantic Drift Detector (TSDD) — enables the system to automatically adapt when new deepfake generation tools emerge, addressing the core generalization problem that plagues all current detection systems.

# 2. Problem Statement

## 2.1 Core Problem

Deepfake technology uses advanced AI techniques — primarily Generative Adversarial Networks (GANs), diffusion models, and neural face-swapping algorithms — to create highly realistic synthetic media that are virtually indistinguishable from authentic content by the human eye. The deployment of such content on social media platforms at scale poses an unprecedented threat to information integrity, public trust, and individual safety.

## 2.2 Specific Challenges

- **Weaponization:** Deepfakes can be weaponized for political misinformation, financial fraud, identity theft, and non-consensual media creation.
- **Human limitations:** Human moderators cannot reliably detect high-quality deepfakes, and the volume of content on social media is far too large for manual review.
- **System brittleness:** Existing automated detection systems fail on deepfakes produced by new or unseen generation tools, requiring expensive retraining cycles.
- **Lack of real-time capability:** Most detection tools operate only on static images or pre-recorded videos and cannot process live or streaming content in real time.
- **Single modality bias:** Current systems focus on a single modality (visual only) and miss cross-modal inconsistencies such as audio-visual desync.

## 2.3 Identified Research Gaps

| Research Gap | Description |
|---|---|
| **Generalization failure** | Models trained on one dataset (e.g., FF++) fail on deepfakes created by newer tools, as they memorize artifact patterns rather than learning generalizable features. |
| **High-quality deepfake resistance** | State-of-the-art generation tools (StyleGAN3, Stable Diffusion) produce outputs that defeat most existing detection models. |
| **No real-time detection** | Existing systems are computationally heavy and are designed for offline batch processing, not live social media stream monitoring. |
| **No multimodal fusion** | Most systems analyze only visual frames. Audio-visual sync errors, unnatural speech patterns, and metadata anomalies are ignored. |
| **Robustness to post-processing** | JPEG compression, resizing, and noise addition — common on social media — degrade features and fool detectors. |
| **No explainability** | Detection verdicts are black-box outputs with no indication of which region or feature caused the classification, limiting forensic utility. |
| **No adaptive learning** | Existing systems are static — they do not update when the distribution of deepfakes shifts due to new generation methods. |

# 3. Proposed Solution — MFAD-Net

## 3.1 Solution Overview

MFAD-Net is a real-time, multimodal deepfake detection system purpose-built for social media monitoring. It addresses every identified research gap through a combination of proven deep learning components and a novel adaptive module. The system ingests video, audio, and post metadata simultaneously, fuses these modalities through a cross-modal attention mechanism, and produces an interpretable verdict with a confidence score and visual explanation.

## 3.2 Key Innovations

- **Trimodal fusion (unique):** No existing detection system simultaneously processes video frames, audio signals, and post metadata. MFAD-Net's Cross-Modal Attention Fusion (CMAF) module learns cross-modality inconsistencies — such as audio-visual desync and metadata anomalies — that are invisible to single-modality systems.
- **Temporal-Semantic Drift Detector (unique):** The TSDD module monitors the live distribution of incoming media using Maximum Mean Discrepancy (MMD). When it detects that the incoming content no longer matches the training distribution — indicating a new deepfake tool is in use — it triggers an automatic lightweight fine-tuning cycle. This is the first application of continual learning to deepfake detection.
- **GAN fingerprint memory (unique):** A GAN fingerprint memory stores artifact signatures for each known generation tool. New detections are matched against this memory, allowing the system to identify not just that a media is fake, but which tool likely generated it.
- **Per-prediction explainability:** GradCAM heatmaps and LIME audio attribution are generated for every prediction, highlighting the exact facial regions or audio segments that triggered the detection — making the system suitable for forensic and legal use.

## 3.3 Gap Resolution Summary

| Gap | MFAD-Net Solution |
|---|---|
| **Generalization failure** | Three-dataset training (FF++, DFDC, WildDeepfake) + TSDD self-adaptation to new distributions |
| **High-quality deepfake resistance** | FFT frequency branch captures invisible spectral artifacts that spatial-only models miss |
| **No real-time detection** | EfficientNet-B4 backbone chosen for speed; TSDD runs as a parallel async process |
| **No multimodal fusion** | CMAF module fuses video, audio (Wav2Vec2), and metadata (Graph encoder) in a shared transformer |
| **Robustness to post-processing** | Frequency domain analysis is robust to JPEG compression and resizing |
| **No explainability** | GradCAM + LIME generate visual and audio attribution maps per prediction |
| **No adaptive learning** | TSDD detects distribution shift via MMD and triggers automatic fine-tune cycles |

# 4. Dataset Selection

## 4.1 Strategy

A critical weakness of all existing deepfake detection systems is training on a single dataset, which leads to overfitting to that dataset's specific artifacts. MFAD-Net employs a three-dataset ensemble training strategy that forces the model to learn generalizable features rather than dataset-specific patterns.

## 4.2 Selected Datasets

| Dataset | Contents | Why Selected |
|---|---|---|
| **FaceForensics++** | 1,000 videos, 5 manipulation methods (Deepfakes, Face2Face, FaceShift, NeuralTextures, FaceSwap). High-quality and compressed variants. | Baseline. Well-studied benchmark. Provides clean labeled ground truth for all 5 manipulation types. |
| **DFDC (DeepFake Detection Challenge)** | 100,000+ videos from Facebook AI. Diverse races, ages, lighting conditions, compression levels, and backgrounds. | Real-world diversity. Trains robustness to production social media conditions. |
| **WildDeepfake** | 7,314 face sequences scraped directly from the internet — not from controlled lab generation. | In-the-wild generalization. Closest to actual social media content. Most underused dataset in literature. |

## 4.3 Data Splits

| Split | Details |
|---|---|
| **Training set** | 70% of combined dataset (~80,000 samples) |
| **Validation set** | 15% for hyperparameter tuning (~17,000 samples) |
| **Test set** | 15% held out, never seen during training (~17,000 samples) |
| **TSDD prototype memory** | 1,000 samples per class for distribution prototype initialization |

# 5. System Architecture

## 5.1 Architecture Overview

MFAD-Net is organized into five functional layers: Input Ingestion, Feature Extraction, Cross-Modal Fusion, Adaptive Drift Detection, and Output Generation. Each layer is implemented as an independent module with well-defined interfaces, enabling modular testing and replacement.

## 5.2 Layer-by-Layer Description

### Layer 1 — Input Ingestion

The system accepts three input streams from a social media post:

- **Visual stream:** Video frames: Extracted at 10 FPS. Each frame is face-detected and cropped using MTCNN, then resized to 224×224 pixels.
- **Audio stream:** Audio signal: Extracted from the video using FFmpeg. Converted to 16kHz mono WAV. Mel-frequency cepstral coefficients (MFCCs) computed in parallel.
- **Metadata stream:** Post metadata: Platform name, posting timestamp, account age, follower count, sharing velocity, geographic spread of engagement.

### Layer 2 — Feature Extraction

Three parallel feature extraction branches process each modality:

- **Visual branch:** EfficientNet-B4 pretrained on ImageNet extracts spatial features. A parallel FFT branch computes the 2D Fourier transform and extracts spectral magnitude features — these capture GAN fingerprints invisible to spatial models. Both outputs are concatenated into a 1792-dimensional visual feature vector.
- **Audio branch:** Facebook's Wav2Vec 2.0 (pretrained on LibriSpeech) extracts voice embeddings. MFCC features (40 coefficients) are concatenated to capture spectral speech patterns. Output: 768-dimensional audio feature vector.
- **Metadata branch:** A Graph Attention Network (GAT) encodes the post's propagation pattern as a graph where nodes are accounts and edges are sharing relationships. Output: 256-dimensional metadata feature vector.

### Layer 3 — Cross-Modal Attention Fusion (CMAF)

The three feature vectors are projected to a common 512-dimensional embedding space and fed into a shared 4-head transformer encoder. The transformer learns cross-modal attention — specifically identifying inconsistencies between what the face looks like, what the voice sounds like, and how the post is spreading. Two auxiliary outputs are produced: an audio-visual sync score (quantifying lip-sync error) and a metadata anomaly score.

### Layer 4 — Temporal-Semantic Drift Detector (TSDD)

This is the novel core module of MFAD-Net. It operates in parallel with the main inference pipeline:

- Maintains a rolling window of the last 500 analyzed samples in the feature space.
- Computes Maximum Mean Discrepancy (MMD) between the current window distribution and the stored prototype distribution of known real/fake content.
- When MMD exceeds a threshold (indicating a new deepfake generation tool is producing previously unseen artifacts), a lightweight fine-tuning cycle is triggered using the new samples.
- A GAN fingerprint memory stores cluster centroids for each known generation tool. New samples are matched to the nearest centroid to identify the probable generation source.

### Layer 5 — Output Generation

The system produces four outputs per analyzed post:

- **Verdict:** A verdict (Real / Fake / Uncertain) with a calibrated probability score.
- **Visual explanation:** A GradCAM heatmap overlaid on the video frame showing which facial regions triggered detection.
- **Audio explanation:** LIME audio attribution showing which audio segments were anomalous.
- **Drift alert:** A drift alert if the TSDD detects a new generation tool, along with the probable tool signature.

# 6. Core Algorithm — MFAD-Net

## 6.1 Full Algorithm (Pseudocode)

```
INPUT:  Social media post P = { V (video), A (audio), M (metadata) }
OUTPUT: { verdict, P(fake), confidence_interval, heatmap, drift_flag }

═══════════════════════════════════════════════════════════
STEP 1 — PREPROCESSING
═══════════════════════════════════════════════════════════
  faces   = MTCNN_detect(V)                // Extract face crops
  frames  = resize(faces, 224x224)          // Normalize size
  audio   = FFmpeg_extract(V)               // Extract audio
  wav     = resample(audio, 16000Hz)        // Standardize rate

═══════════════════════════════════════════════════════════
STEP 2 — FEATURE EXTRACTION (parallel)
═══════════════════════════════════════════════════════════
  F_spatial  = EfficientNet_B4(frames)      // 1280-dim
  F_freq     = FFT_branch(frames)           // 512-dim spectral
  F_visual   = concat(F_spatial, F_freq)    // 1792-dim

  F_voice    = Wav2Vec2(wav)                // 768-dim
  F_mfcc     = MFCC(wav, n_coef=40)         // 40-dim
  F_audio    = concat(F_voice, F_mfcc)      // 808-dim

  F_meta     = GraphAttentionNet(M)         // 256-dim

═══════════════════════════════════════════════════════════
STEP 3 — CROSS-MODAL FUSION (CMAF)
═══════════════════════════════════════════════════════════
  F_v_proj   = Linear(F_visual, 512)        // Project to 512
  F_a_proj   = Linear(F_audio, 512)
  F_m_proj   = Linear(F_meta, 512)
  F_fused    = Transformer([F_v, F_a, F_m], heads=4)
  score_av   = AV_sync_loss(F_v_proj, F_a_proj) // Lip-sync
  score_meta = anomaly_score(F_m_proj)

═══════════════════════════════════════════════════════════
STEP 4 — DRIFT DETECTION (TSDD) — async parallel process
═══════════════════════════════════════════════════════════
  window     = FIFO_queue(F_fused, size=500)
  mmd_score  = MMD(window, prototype_distribution)

  IF mmd_score > DRIFT_THRESHOLD:
    drift_flag = TRUE
    fine_tune(model, new_flagged_samples, lr=1e-5, epochs=3)
    update_prototype_memory(window)
    fingerprint = cluster_new_artifacts(window)
    GAN_memory.append(fingerprint)
  ELSE:
    drift_flag = FALSE
    tool_match = nearest_neighbor(F_fused, GAN_memory)

═══════════════════════════════════════════════════════════
STEP 5 — CLASSIFICATION
═══════════════════════════════════════════════════════════
  logits     = Classifier(F_fused + drift_signal)
  P_fake     = Softmax(logits)[1]           // Fake probability
  CI         = MonteCarloDrop(model, F_fused, passes=50)

  IF P_fake > 0.75: verdict = FAKE
```

```
  ELIF P_fake < 0.35: verdict = REAL
  ELSE: verdict = UNCERTAIN

═══════════════════════════════════════════════════════════
STEP 6 — EXPLAINABILITY
═══════════════════════════════════════════════════════════
  heatmap    = GradCAM(model, frames, target_layer='block7')
  audio_attr = LIME(F_audio, num_features=10)

RETURN { verdict, P_fake, CI, heatmap, audio_attr, drift_flag }
```

## 6.2 TSDD — Drift Detection Detail

The Maximum Mean Discrepancy (MMD) between two distributions P and Q is defined as:

$$MMD^2(P, Q) = E[k(x,x')] - 2 \cdot E[k(x,y)] + E[k(y,y')]$$

where k is a Gaussian RBF kernel and x ~ P, y ~ Q. This measures whether two sets of samples come from the same distribution. When the incoming window of feature vectors diverges significantly from the stored prototype (MMD > 0.15 empirically), the system infers that a previously unseen generation method is producing the new content.

# 7. System Workflow

## 7.1 End-to-End Flow

The following describes the complete journey of a social media post through the MFAD-Net pipeline from input to verdict:

1. **Input submission:** A social media post URL or media file is submitted to the system via the REST API or the real-time WebSocket stream monitor.
2. **Media parsing:** FFmpeg extracts video frames at 10 FPS and the audio track. MTCNN detects and crops all faces in each frame.
3. **Parallel feature extraction:** The three feature extraction branches (visual, audio, metadata) run in parallel on GPU, producing three feature vectors.
4. **Fusion:** The CMAF transformer fuses all three vectors, computing cross-modal attention weights and auxiliary sync/anomaly scores.
5. **Drift check:** The TSDD module asynchronously checks the current batch against the prototype distribution. If drift is detected, a fine-tuning job is queued.
6. **Classification:** The classifier head produces a probability score. Monte Carlo dropout provides a 95% confidence interval.
7. **Explanation generation:** GradCAM generates a heatmap. LIME generates audio attribution. Both are stored alongside the verdict.
8. **Result delivery:** The verdict, confidence, heatmap URL, and any drift alerts are returned via the API and displayed on the monitoring dashboard.

## 7.2 Real-Time Monitoring Flow

For continuous social media monitoring, the system operates as follows:

- A social media scraper module polls configured accounts and hashtags every 60 seconds using platform APIs.
- New posts are queued in a Redis message queue and processed by worker instances in parallel.
- High-confidence fake verdicts trigger an immediate alert to the monitoring dashboard and are logged to the database.
- The TSDD drift monitor runs as a background process, checking distribution every 500 processed items.
- The system maintains a throughput target of 50 posts per minute on a single T4 GPU, sufficient for monitoring up to 3 social media accounts continuously.

# 8. Implementation Plan

## 8.1 Technology Stack

| Component | Technology |
|---|---|
| **Deep Learning Framework** | PyTorch 2.x with CUDA 11.8 |
| **Visual backbone** | EfficientNet-B4 (timm library) |
| **Audio encoder** | Wav2Vec 2.0 (HuggingFace Transformers) |
| **Graph encoder** | PyTorch Geometric (GAT) |
| **Face detection** | MTCNN (facenet-pytorch) |
| **Explainability** | pytorch-grad-cam, LIME |
| **Backend API** | FastAPI with WebSocket support |
| **Frontend Dashboard** | React.js with Recharts |
| **Message Queue** | Redis (for real-time stream) |
| **Database** | PostgreSQL (verdicts), Redis (cache) |
| **Training Platform** | Google Colab Pro / Local GPU (CUDA) |
| **Deployment** | Docker + Docker Compose |

## 8.2 Project Folder Structure

```
mfad-net/
├── data/
│   ├── raw/              # Downloaded datasets
│   ├── processed/        # Preprocessed face crops
│   └── splits/           # Train/val/test CSV files
├── models/
│   ├── visual_encoder.py # EfficientNet + FFT branch
│   ├── audio_encoder.py  # Wav2Vec2 + MFCC branch
│   ├── meta_encoder.py   # Graph attention network
│   ├── cmaf.py           # Cross-modal attention fusion
│   ├── tsdd.py           # Drift detector (UNIQUE MODULE)
│   └── mfad_net.py       # Full assembled model
├── training/
│   ├── train.py          # Main training script
│   ├── evaluate.py       # Evaluation + metrics
│   └── config.yaml       # Hyperparameters
├── backend/
│   ├── main.py           # FastAPI app
│   ├── inference.py      # Prediction pipeline
│   ├── explainability.py # GradCAM + LIME
│   └── scraper.py        # Social media monitor
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── Dashboard.jsx
│   │   └── components/
│   └── package.json
├── weights/              # Saved model weights (.pth)
```

```
├── requirements.txt
├── docker-compose.yml
└── README.md
```

# 9. Project Phases & Timeline

The project is divided into eight phases, designed to be executed sequentially with clear deliverables at each stage. Total estimated duration: 3–4 weeks for a fully working system.

### Phase 0: Environment Setup [Day 1 — ~2 hours]

- Install Python 3.10, PyTorch 2.x, CUDA toolkit
- Set up Google Colab Pro account (free tier sufficient for training)
- Create project folder structure
- Install all dependencies from requirements.txt
- Verify GPU availability with a simple PyTorch CUDA test
- Deliverable: Working Python environment ready for development

### Phase 1: Dataset Download & Preprocessing [Days 2–4 — ~6 hours work]

- Register and download FaceForensics++ (requires academic request form)
- Download DFDC dataset from Kaggle (~470GB, use subset of 20k videos)
- Download WildDeepfake from GitHub repository
- Run face detection (MTCNN) on all videos to extract 224x224 face crops
- Build balanced dataset: 50% real, 50% fake across all three sources
- Generate train/val/test split CSV files
- Deliverable: Preprocessed face crop dataset ready for training (~50GB)

### Phase 2: Model Building — Feature Extractors [Days 5–7 — ~8 hours work]

- Implement visual_encoder.py: EfficientNet-B4 + parallel FFT frequency branch
- Implement audio_encoder.py: Wav2Vec 2.0 fine-tuned + MFCC extraction
- Implement meta_encoder.py: Graph Attention Network for metadata
- Unit test each encoder independently with dummy inputs
- Verify output dimensions: visual=1792, audio=808, metadata=256
- Deliverable: Three working encoder modules with tested forward passes

### Phase 3: Fusion Module + TSDD (Unique Module) [Days 8–11 — ~10 hours work]

- Implement cmaf.py: Shared 4-head transformer encoder with modality-aware masking
- Implement AV sync loss auxiliary head and metadata anomaly scorer
- Implement tsdd.py: MMD computation, rolling window, fine-tune trigger
- Implement GAN fingerprint memory with nearest-neighbour matching
- Assemble complete mfad_net.py combining all modules
- End-to-end forward pass test with a sample video
- Deliverable: Full MFAD-Net model assembled and producing predictions

### Phase 4: Training (Runs on Colab/GPU) [Days 12–14 — runs overnight]

- Configure train.py with all hyperparameters (lr=1e-4, batch=32, epochs=20)

- Upload to Google Colab, connect T4 GPU runtime
- Run training — estimated 8–12 hours on Colab free tier
- Monitor validation accuracy every epoch, paste results for tuning
- Save best model checkpoint as weights/mfad_net_best.pth
- Target: >92% accuracy, >0.95 AUC on held-out test set
- Deliverable: Trained model weights file ready for deployment

## Phase 5: Backend API (FastAPI + WebSocket) [Days 15–17 — ~6 hours work]

- Implement FastAPI app with POST /analyze endpoint (file upload)
- Implement WebSocket endpoint for real-time stream monitoring
- Build inference.py pipeline: load weights → preprocess → predict → return JSON
- Implement explainability.py: GradCAM heatmap generation and LIME audio attribution
- Implement scraper.py: polling social media URLs on a configurable schedule
- Test API with Postman / curl, verify sub-2-second response time
- Deliverable: Working REST API serving predictions with explanations

## Phase 6: Frontend Dashboard (React) [Days 18–20 — ~8 hours work]

- Build main Dashboard.jsx with video upload and URL input components
- Implement real-time verdict display: verdict badge, confidence bar, heatmap overlay
- Build drift alert panel showing TSDD notifications
- Implement per-modality confidence breakdown chart (using Recharts)
- Connect to backend WebSocket for live monitoring feed
- Responsive design for desktop and tablet
- Deliverable: Interactive web dashboard showing live deepfake detection results

## Phase 7: Testing, Evaluation & Viva Preparation [Days 21–25]

- Run full evaluation on held-out test set: accuracy, AUC-ROC, F1, confusion matrix
- Benchmark inference speed: measure average latency per video
- Prepare evaluation charts: accuracy vs epoch, ROC curve, per-class breakdown
- Record a 3-minute demo video showing end-to-end detection
- Prepare viva Q&A document covering all expected questions
- Package all code, weights, and documentation for submission
- Deliverable: Complete working system + evaluation report + demo video

# 10. Expected Outcomes & Evaluation Metrics

## 10.1 Performance Targets

| Metric | Target Value |
|---|---|
| **Detection Accuracy** | > 92% on combined test set |
| **AUC-ROC Score** | > 0.95 |
| **F1 Score** | > 0.91 |
| **Inference Latency** | < 2 seconds per video on T4 GPU |
| **Drift Detection Rate** | Alert within 500 new samples of a new generation tool |
| **Throughput** | > 50 posts per minute in real-time monitoring mode |

## 10.2 Deliverables

- Trained MFAD-Net model weights (mfad_net_best.pth)
- Complete source code (all 8 modules) with documentation
- FastAPI backend serving predictions via REST and WebSocket
- React frontend dashboard with heatmap visualization
- Evaluation report with accuracy charts, confusion matrix, ROC curve
- 3-minute video demonstration of the live system
- Project report (this document) and viva Q&A preparation guide

# References

1. Rossler, A., et al. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. ICCV 2019.

2. Dolhansky, B., et al. (2020). The DeepFake Detection Challenge (DFDC) Dataset. arXiv:2006.07397.

3. Zi, B., et al. (2020). WildDeepfake: A Challenging Real-World Dataset for Deepfake Detection. ACM MM 2020.

4. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.

5. Baevski, A., et al. (2020). Wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS 2020.

6. Gretton, A., et al. (2012). A Kernel Two-Sample Test. Journal of Machine Learning Research, 13(1).

7. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. ICCV 2017.

8. Ribeiro, M. T., et al. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. KDD 2016.